1. Problem + Algorithm

Experiment No.: 08

Title: Simulated Annealing for Function Optimization

Aim:

Implement Simulated Annealing to minimize the given objective function.

Problem Statement:

Minimize the objective function **$f(x) = x^2 + 10 \sin(x)$** using a temperature-controlled stochastic search.

Algorithm:

- Initialize solution.
- Set initial temperature.
- Generate neighboring solution.
- Calculate change in objective function.
- Accept/reject according to probability.
- Reduce temperature.
- Repeat until stopping criterion.
- Return best solution.

2. Program

Write/print the program code.

Should contain:

- Important code sections
- Function definitions
- Core algorithm
- Key parameter settings

Key parameter settings

```
neighbour_step = 1
iterations_per_temperature = 1
seed = 42

x0 = float(input('Enter x0: '))
T0 = float(input('Enter T0: '))
alpha = float(input('Enter alpha: '))
T_min = float(input('Enter T_min: '))
```

Function definition

```
def f(x):
return x**2 + 10*np.sin(x)
```

Core Simulated Annealing

```
current_x = x0
current_value = f(current_x)
temperature = T0

while temperature >= T_min:
for _ in range(iterations_per_temperature):
candidate_x = current_x + random.choice(
[-neighbour_step, neighbour_step]
)
candidate_value = f(candidate_x)
delta = candidate_value - current_value

if delta <= 0:
accepted = True
probability = 1.0
else:
probability = math.exp(-delta / temperature)
accepted = random.random() < probability

if accepted:
current_x = candidate_x
current_value = candidate_value

temperature *= alpha
```

The complete executable program can be maintained digitally through Github.

3. Output + Analysis

Objective Function:

$f(x) = x^2 + 10 \sin(x)$

Search Interval:

$-10 \leq x \leq 10$

Initial Solution: x0 = 2.5

Initial Temperature: T0 = 50

Cooling Rate: 0.60

Maximum Iterations: 22 (T_min = 0.001)

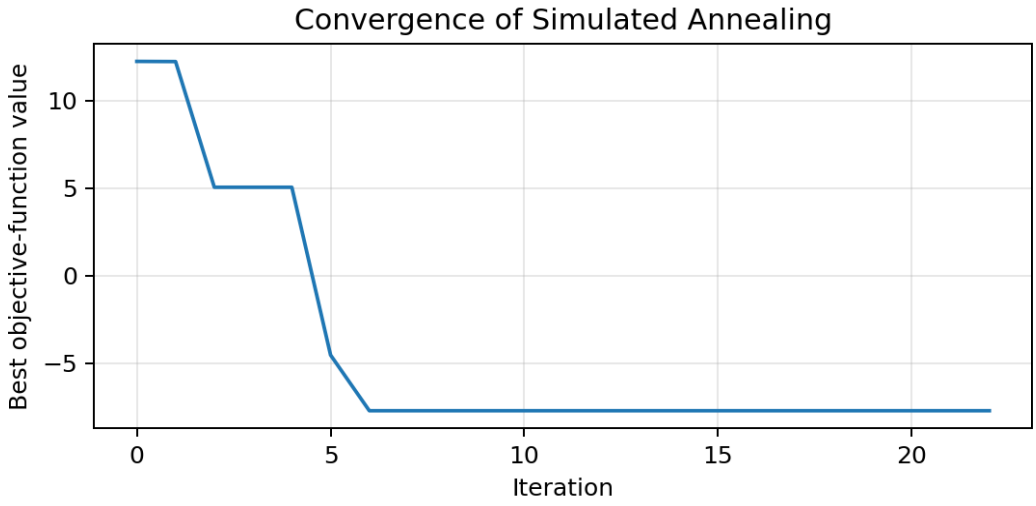
Program Execution

Final Optimal Solution: $x^* = -1.5000$

Minimum Function Value: $f(x^*) = -7.7249$

Execution Status: Successfully Executed

Convergence Graph



X-axis → Iteration

Y-axis → Best objective-function value

The graph should demonstrate how the solution approaches the optimum.

Parameter Modification

S. No.	Parameter	Original	Modified	Original Result	Modified Result	Observation
1	Initial Temperature T0	50	100	x = –1.5000	x = –1.0000	Effect on exploration
2	Cooling Rate α	0.60	0.95	x = –1.5000	x = –1.0000	Effect on convergence
3	Maximum Iterations	22	225	f(x) = –7.7249	f(x) = –7.4147	Effect on optimization
4	Initial Solution x0	2.5	8	x = –1.5000	x = –1.0000	Effect of starting point

1. Problem + Algorithm

Experiment No.: 09

Title: Genetic Algorithm for Traveling Salesman Problem

Aim:

Implement Genetic Algorithm (GA) for solving the Traveling Salesman Problem (TSP).

Problem Statement:

Find a short closed tour that visits every city exactly once and returns to the starting city. The objective is to minimize total tour distance.

Algorithm:

- Initialize a population of valid city permutations.
- Calculate route distance and fitness.
- Select parents using tournament selection.
- Apply ordered crossover to create offspring.
- Apply swap mutation.
- Keep the best individuals using elitism.
- Repeat for the specified number of generations.
- Return the best route found.

2. Program

Write/print the program code.

Should contain:

- Important code sections
- Function definitions
- Core algorithm
- Key parameter settings

Key parameter settings

```
population_size = read_int('Population size', 100, 4)
generations = read_int('Number of generations', 300, 1)
crossover_rate = read_probability('Crossover rate', 0.9)
mutation_rate = read_probability('Mutation rate', 0.15)
```

Function definitions

```
def route_distance(route):
total = 0.0
for i in range(len(route)):
current_city = coordinates[route[i]]
next_city = coordinates[route[(i + 1) % len(route)]]
total += math.dist(current_city, next_city)
return total

def fitness(route):
distance = route_distance(route)
return 1.0 / distance if distance > 0 else float('inf')
```

Selection, crossover and mutation

```
def tournament_selection(population, tournament_size=3):
contestants = random.sample(
population, min(tournament_size, len(population))
)
return min(contestants, key=route_distance).copy()

def ordered_crossover(parent1, parent2):
size = len(parent1)
left, right = sorted(random.sample(range(size), 2))
child = [None] * size
child[left:right + 1] = parent1[left:right + 1]
remaining = [gene for gene in parent2 if gene not in child]
fill_positions = list(range(right + 1, size)) + list(range(0, left))
for position, gene in zip(fill_positions, remaining):
child[position] = gene
return child

def swap_mutation(route):
i, j = random.sample(range(len(route)), 2)
route[i], route[j] = route[j], route[i]
return route
```

Core GA loop

```
population = [
random.sample(range(n_cities), n_cities)
for _ in range(population_size)
]

for generation in range(generations):
population.sort(key=route_distance)
next_population = [route.copy() for route in population[:elite_count]]

while len(next_population) < population_size:
parent1 = tournament_selection(population)
parent2 = tournament_selection(population)

if random.random() < crossover_rate:
child = ordered_crossover(parent1, parent2)
else:
child = parent1.copy()

if random.random() < mutation_rate:
child = swap_mutation(child)

next_population.append(child)

population = next_population
```

The complete executable program can be maintained digitally through Github.

3. Output + Analysis

Objective Function:

Minimize total tour distance; fitness = 1 / total distance

Search Interval:

Cities A, B, C, D, E, F, G, H

Initial Solution: Population = 100

Crossover Rate: 0.90

Mutation Rate: 0.15

Maximum Generations: 300

Program Execution

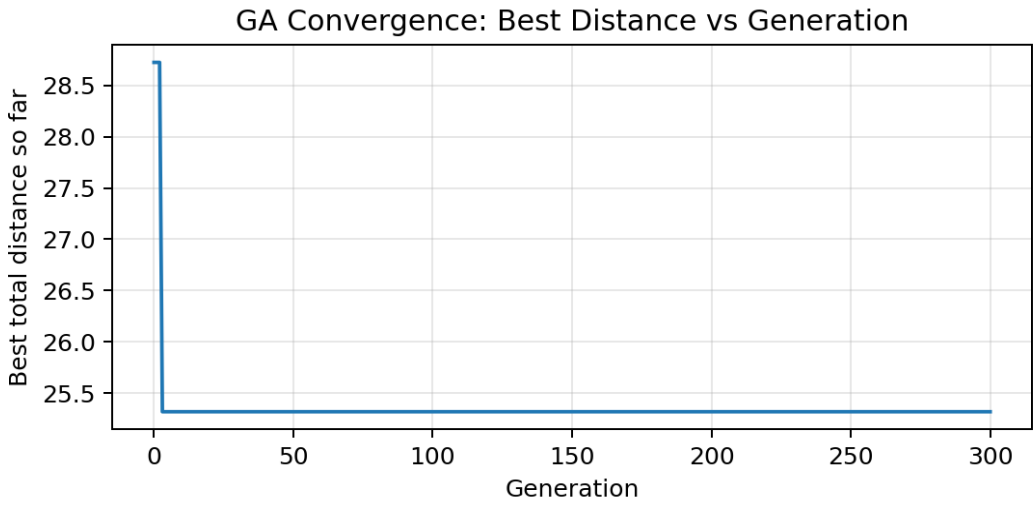
Best Route: A → G → D → E → H → C → B → F → A

Minimum Total Distance: 25.319

Best Fitness: 0.039496

Execution Status: Successfully Executed

Convergence Graph



X-axis → Generation

Y-axis → Best total distance

The graph should demonstrate how the solution approaches the optimum.

Parameter Modification

S. No.	Parameter	Original	Modified	Original Result	Modified Result	Observation
1	Population Size	100	50	Distance = 25.319	Distance = 25.319	Effect on population diversity
2	Generations	300	500	Distance = 25.319	Distance = 25.319	Effect on convergence
3	Crossover Rate	0.90	0.80	Route length = 25.319	Route length = 25.319	Effect on recombination
4	Mutation Rate	0.15	0.25	Fitness = 0.039496	Fitness = 0.039496	Effect on exploration

1. Problem + Algorithm

Experiment No.: 10

Title: Particle Swarm Optimization for Function Optimization

Aim:

Implement Particle Swarm Optimization (PSO) to minimize the Rosenbrock objective function.

Problem Statement:

Minimize the two-variable Rosenbrock function using a swarm of particles that update their positions and velocities from personal and global best solutions.

Algorithm:

- Initialize particle positions and velocities.
- Evaluate the objective function for every particle.
- Set each particle's personal best.
- Set the global best particle.
- Update velocity using inertia, personal and global terms.
- Update and bound particle positions.
- Update personal and global best values.
- Repeat until the maximum number of iterations.
- Return the global best solution.

2. Program

Write/print the program code.

Should contain:

- Important code sections
- Function definitions
- Core algorithm
- Key parameter settings

Key parameter settings

```
n_particles = 30
lower_bounds = np.array([-2.0, -1.0])
upper_bounds = np.array([2.0, 3.0])

w = 0.7
c1 = 1.5
c2 = 1.5
max_iterations = 100
```

Objective function

```
def rosenbrock(position):
x, y = position
return (1 - x)**2 + 100 * (y - x**2)**2
```

Initialization and best values

```
positions = np.random.uniform(
lower_bounds, upper_bounds, (n_particles, 2)
)
velocities = np.random.uniform(-0.2, 0.2, (n_particles, 2))

pbest_positions = positions.copy()
pbest_fitness = np.array([rosenbrock(p) for p in positions])

gbest_index = np.argmin(pbest_fitness)
gbest_position = pbest_positions[gbest_index].copy()
gbest_fitness = pbest_fitness[gbest_index]
```

Core PSO update

```
for iteration in range(max_iterations):
r1 = np.random.rand(n_particles, 2)
r2 = np.random.rand(n_particles, 2)

velocities = (
w * velocities
+ c1 * r1 * (pbest_positions - positions)
+ c2 * r2 * (gbest_position - positions)
)

positions = positions + velocities
positions = np.clip(positions, lower_bounds, upper_bounds)

fitness_values = np.array([rosenbrock(p) for p in positions])

improved = fitness_values < pbest_fitness
pbest_positions[improved] = positions[improved]
pbest_fitness[improved] = fitness_values[improved]

gbest_index = np.argmin(pbest_fitness)
if pbest_fitness[gbest_index] < gbest_fitness:
gbest_position = pbest_positions[gbest_index].copy()
gbest_fitness = pbest_fitness[gbest_index]
```

The complete executable program can be maintained digitally through Github.

3. Output + Analysis

Objective Function:

$f(x,y) = (1-x)^2 + 100(y-x^2)^2$

Search Interval:

$-2 \le x \le 2, -1 \le y \le 3$

Initial Solution: 30 particles

Inertia Weight w = 0.70

c1 = 1.50, c2 = 1.50

Maximum Iterations: 100

Program Execution

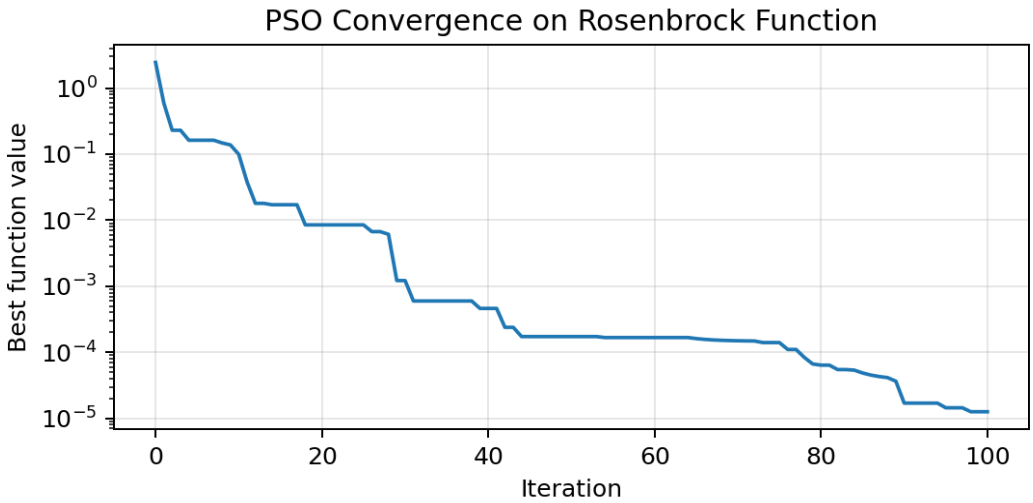
Best Position: [0.99645787, 0.99295397]

Minimum Function Value: 1.2613 × 10■■

Theoretical Minimum: [1, 1], f = 0

Execution Status: Successfully Executed

Convergence Graph



X-axis → Iteration

Y-axis → Best objective-function value

The graph should demonstrate how the solution approaches the optimum.

Parameter Modification

S. No.	Parameter	Original	Modified	Original Result	Modified Result	Observation
1	Number of Particles	30	20	f(x,y) = 1.2613e–5	f(x,y) = 1.1506e–6	Effect on swarm diversity
2	Inertia Weight w	0.70	0.50	Best ≈ [0.9965,0.9930]	Best ≈ [1.0011,1.0021]	Effect on movement
3	Cognitive c1	1.50	2.00	f = 1.2613e–5	f = 1.1506e–6	Effect of personal learning
4	Maximum Iterations	100	150	f = 1.2613e–5	f = 1.1506e–6	Effect on optimization

1. Problem + Algorithm

Experiment No.: 11

Title: Ant Colony Optimization for Traveling Salesman Problem

Aim:

Implement Ant Colony Optimization (ACO) for finding a short tour through the given cities.

Problem Statement:

Find a short closed tour through A, B, C, D and E using pheromone trails and distance-based desirability.

Algorithm:

- Initialize the distance matrix and pheromone trails.
- Calculate desirability $\eta ij = 1 / dij$.
- Calculate transition probabilities from pheromone and desirability.
- Construct tours for the ants.
- Calculate each tour length.
- Calculate pheromone deposits.
- Evaporate old pheromone.
- Update pheromone trails.
- Repeat the process for further iterations.

2. Program

Write/print the program code.

Should contain:

- Important code sections
- Function definitions
- Core algorithm
- Key parameter settings

Key parameter settings

```
alpha = 1
beta = 2
rho = 0.5
Q = 100
start_cities = [0, 1, 2]

pheromone = np.ones_like(distance)
np.fill_diagonal(pheromone, 0)
```

Function definition

```
def transition_probabilities(current, unvisited, pheromone):
weights = (
pheromone[current, unvisited]**alpha
* eta[current, unvisited]**beta
)
return weights / weights.sum()
```

Tour length and pheromone deposit

```
lengths = []
for ant, tour in enumerate(tours, start=1):
edge_lengths = distance[tour[:-1], tour[1:]]
length = edge_lengths.sum()
lengths.append(length)

lengths = np.array(lengths)
rewards = Q / lengths

deposits = np.zeros_like(distance)
for tour, reward in zip(tours, rewards):
for i, j in zip(tour[:-1], tour[1:]):
deposits[i, j] += reward
deposits[j, i] += reward
```

Core pheromone update

```
evaporated = (1 - rho) * pheromone
updated_pheromone = evaporated + deposits

before = transition_probabilities(
0, [1, 2, 3, 4], pheromone
)
after = transition_probabilities(
0, [1, 2, 3, 4], updated_pheromone
)
```

The complete executable program can be maintained digitally through Github.

3. Output + Analysis

Objective Function:

Minimize total tour length Lk = Σ dij over the closed tour

Search Interval:

Cities A, B, C, D, E

Initial Solution: Pheromone τij = 1 on every valid edge

α = 1, β = 2

Evaporation Rate ρ = 0.50

Q = 100; one complete iteration in the notebook

Program Execution

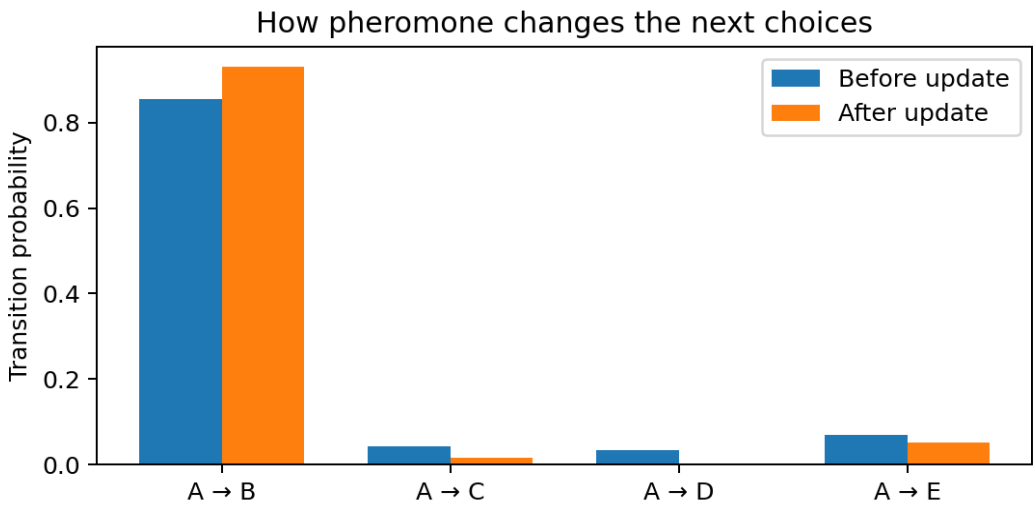
Tour lengths: 27, 28, 28

Best Tour: A → B → E → D → C → A

Best Tour Length: 27

Execution Status: Successfully Executed

Convergence Graph



X-axis → Candidate move from A

Y-axis → Transition probability

The graph should demonstrate how the solution approaches the optimum.

Parameter Modification

S. No.	Parameter	Original	Modified	Original Result	Modified Result	Observation
1	Pheromone Influence α	1	2	$P(A \to B) = 0.930194$	$P(A \to B) \approx 0.995$	Effect on pheromone influence
2	Distance Influence β	2	3	$P(A \to B) = 0.930194$	$P(A \to B) \approx 0.998$	Effect on distance preference
3	Evaporation Rate ρ	0.50	0.30	AB pheromone = 11.3466	AB pheromone = 6.1233	Effect on memory
4	Deposit Constant Q	100	50	Best L = 27	Best L = 27	Effect on deposit strength

1. Problem + Algorithm

Experiment No.: 12
Title: L1, L2 and Elastic Net Regularization for GDP Prediction
Aim:
Implement and compare L1 (Lasso), L2 (Ridge), and Elastic Net regularization for predicting real GDP per capita from macroeconomic indicators.
Problem Statement:
Economic indicators have different scales, correlated predictors, and missing observations. Predict the natural logarithm of GDP per capita and evaluate how regularization changes prediction error, coefficient magnitudes, and feature selection on economies not seen during training. Coefficients are predictive associations, not causal estimates.
Algorithm:

- Retain country-year records with a GDP target and at least six observed predictors.
- Log-transform GDP per capita to reduce right-skewness.
- Split entire economies into training (80%) and testing (20%) groups.
- Impute missing predictor values with training-fold medians and standardize features.
- Use five-fold grouped cross-validation to select regularization parameters.
- Fit Linear Regression, Lasso, Ridge, and Elastic Net; evaluate on held-out economies.
- Vary alpha and compare validation error, coefficient shrinkage, and the number of active predictors.

2. Program

Write/print the main program sections. The code includes the target transformation, a country-level split, pipeline preprocessing, and grouped cross-validation.

Key parameter settings

```
RANDOM_STATE = 42
test_size = 0.20                # split by economy
cv = GroupKFold(n_splits=5)
alpha_grid = np.logspace(-4, 2, 25)
elastic_net_l1_ratio_grid = [0.1, 0.3, 0.5, 0.7, 0.9]
```

Target and leakage-safe split

```
data["log_gdp_per_capita"] = np.log(data[target])
X, y, groups = data[FEATURES], data.log_gdp_per_capita, data.country_code
splitter = GroupShuffleSplit(n_splits=1, test_size=0.20,
                             random_state=RANDOM_STATE)
train_idx, test_idx = next(splitter.split(X, y, groups))
X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
```

Pipeline and regularization search

```
pipeline = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
    ("model", estimator),
])
search = GridSearchCV(pipeline, grid,
    scoring="neg_root_mean_squared_error", cv=cv, n_jobs=-1)
search.fit(X_train, y_train, groups=groups_train)
models[name] = search.best_estimator_
```

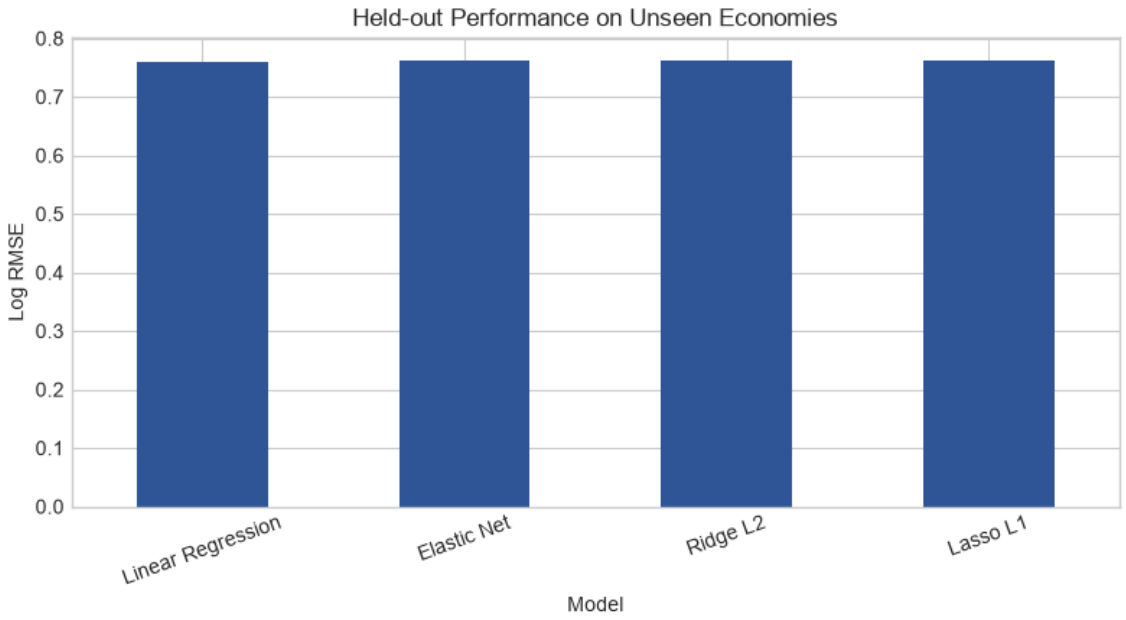
The complete executable program is maintained in the accompanying notebook.

3. Output + Analysis

Objective Function:
Predict log GDP per capita; evaluate with held-out log RMSE, log MAE, and R2.
Dataset and Search Interval:
World Bank World Development Indicators, 2010-2023; 3,283 country-year rows from 241 economies; nine macroeconomic predictors.
Training/Test Split: 2,635 rows from 192 training economies; 648 rows from 49 test economies; zero country overlap.
Cross-validation: Five-fold GroupKFold; alpha grid from 1e-4 to 1e2.
Program Execution
Lowest held-out log RMSE: Linear Regression = 0.7606. Best regularized model by cross-validation: Elastic Net (mean CV log RMSE = 0.9115).

Model	Test log RMSE	Test R2	Nonzero coefficients
Linear Regression	0.7606	0.6537	9
Elastic Net	0.7623	0.6521	9
Ridge L2	0.7623	0.6521	9
Lasso L1	0.7623	0.6521	8

Model Performance Graph



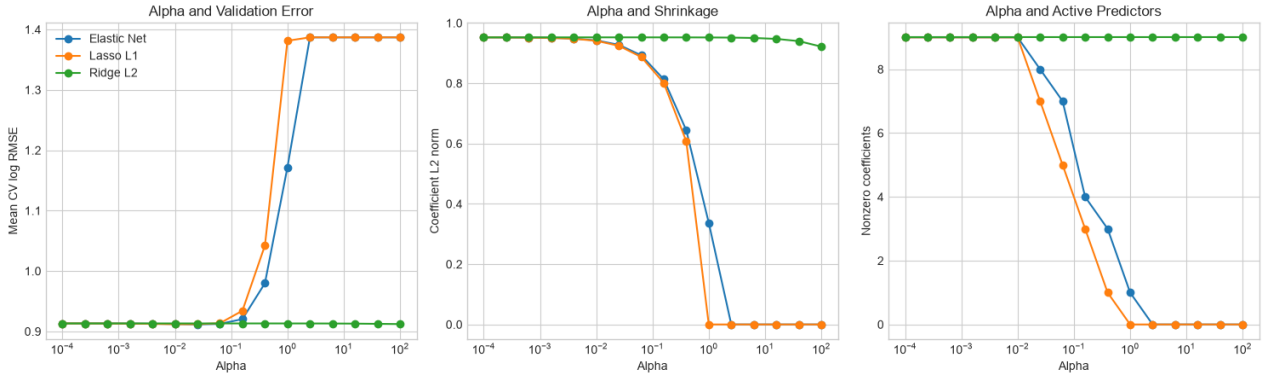
X-axis -> Model; Y-axis -> Held-out log RMSE. The four models have similar test errors; the unregularized baseline is slightly better on this held-out split.

Parameter Modification

S. No.	Method	Alpha	Mean CV log RMSE	Coefficient L2 norm	Nonzero coefficients	Observation
1	Lasso L1	0.01000	0.91195	0.94099	9	L1 penalty can remove predictors as alpha rises.
2	Ridge L2	100.00000	0.91218	0.92177	9	Shrinks coefficient magnitudes while retaining predictors.
3	Elastic Net	0.02512	0.91156	0.92773	8	Lowest CV RMSE among the regularized methods.

Alpha was examined over 1e-4 to 1e2. Stronger penalties shrink coefficients; excessively large penalties increase validation error and may reduce the active feature count.

Alpha Sensitivity



Validation RMSE, coefficient norm, and active predictors show the trade-off between fit and regularization strength.

1. Problem + Algorithm

Experiment No.: 13
Title: Kernel PCA for Nonlinear Income-Group Separation
Aim:
Apply Kernel PCA to macroeconomic data and test whether a nonlinear two-dimensional representation improves the separation of high-income and non-high-income economies.
Problem Statement:
Macroeconomic indicators may have nonlinear relationships. Reduce nine indicators to two components using ordinary PCA and RBF Kernel PCA, then compare both representations with the same balanced logistic-regression classifier. Tune the RBF gamma parameter on training data only and evaluate performance on held-out economies.
Algorithm:

- Select the latest year with at least 150 sufficiently complete classified economies.
- Label high-income economies as 1 and classified low- and middle-income economies as 0.
- Create a stratified 75% training and 25% testing split.
- Median-impute and standardize predictors inside each pipeline.
- Compare nine raw standardized features, two-component PCA, and two-component RBF Kernel PCA.
- Select gamma by five-fold stratified cross-validation using balanced accuracy.
- Report balanced accuracy, macro F1, ROC AUC, confusion matrices, decision boundaries, and gamma sensitivity.

2. Program

Write/print the main program sections. The pipeline compares raw features, ordinary PCA, and RBF Kernel PCA under the same classifier and split.

Key parameter settings

```
selected_year = 2023             # 188 classified economies
FEATURES = ["trade_pct_gdp", "gross_capital_formation_pct_gdp",
            "inflation_consumer_pct", "unemployment_pct",
            "fdi_net_inflows_pct_gdp", "government_consumption_pct_gdp",
            "industry_value_added_pct_gdp", "urban_population_pct",
            "population_growth_pct"]
test_size = 0.25
RANDOM_STATE = 42
n_components = 2
gamma_grid = np.logspace(-3, 1, 13)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

Class definition and split

```
data["high_income"] = (data.income_level_code == "HIC").astype(int)
X, y = data[FEATURES], data.high_income
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=RANDOM_STATE)
```

RBF Kernel PCA pipeline and gamma selection

```
kpca_pipeline = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
    ("reduce", KernelPCA(n_components=2, kernel="rbf",
                         eigen_solver="arpack")),
    ("classifier", LogisticRegression(max_iter=5000,
        class_weight="balanced", random_state=RANDOM_STATE)),
])
search = GridSearchCV(kpca_pipeline, {"reduce__gamma": gamma_grid},
    scoring="balanced_accuracy", cv=cv, n_jobs=-1,
    return_train_score=True)
search.fit(X_train, y_train)
best_gamma = search.best_params_["reduce__gamma"]
```

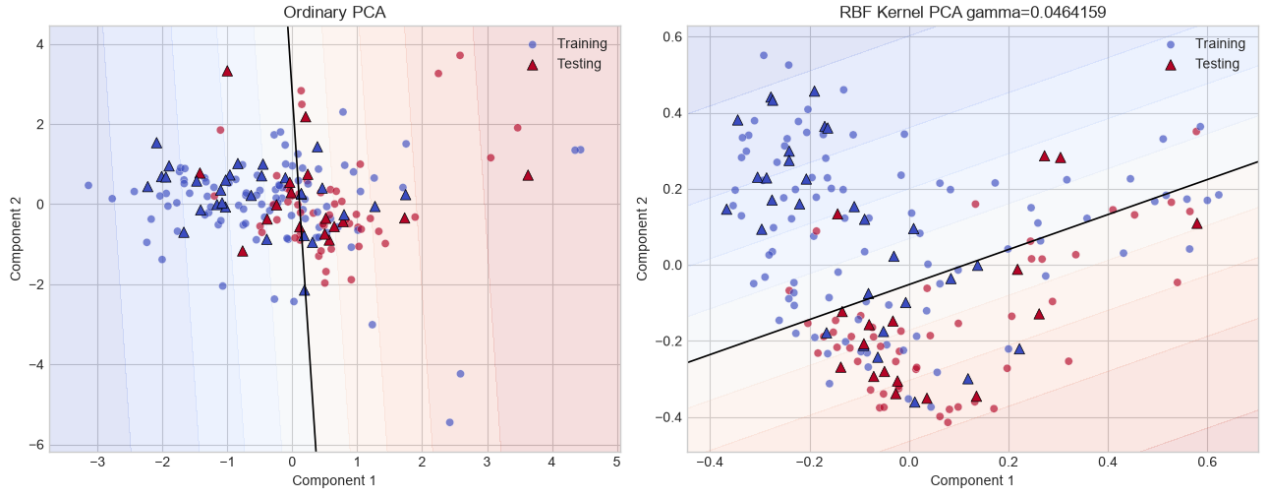
The complete executable program is maintained in the accompanying notebook.

3. Output + Analysis

Objective Function:
Classify high-income versus non-high-income economies; maximize balanced accuracy.
Search Interval:
2023 World Bank cross-section; 188 economies and nine macroeconomic predictors. GDP per capita is excluded from predictors.
Initial Split: 141 training economies and 47 testing economies; stratified by income class.
Selected Parameter: RBF gamma = 0.0464159; mean five-fold CV balanced accuracy = 0.7692.
Program Execution
RBF Kernel PCA improved on ordinary PCA, but did not outperform the nine-feature baseline.

Method	Balanced accuracy	Macro F1	ROC AUC
Raw standardized features	0.8157	0.8157	0.8706
RBF Kernel PCA	0.7451	0.7188	0.8098
Ordinary PCA	0.6147	0.6131	0.7373

Two-Dimensional Representations and Decision Boundaries



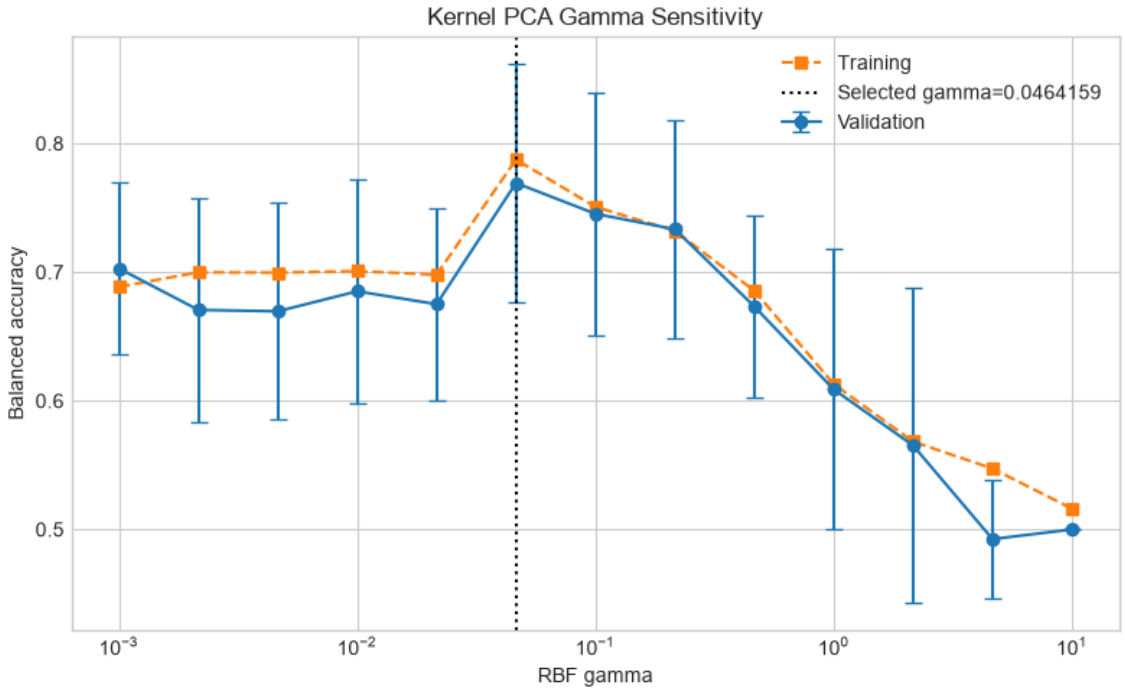
The RBF kernel boundary separates some nonlinear structure better than ordinary PCA, yet compressing to two components loses information compared with the raw nine-feature model.

Parameter Modification

Gamma	Mean train balanced accuracy	Mean CV balanced accuracy	CV standard deviation	Observation
0.0010	0.6885	0.7029	0.0671	Low gamma gives a smoother representation.
0.0464	0.7874	0.7692	0.0925	Selected: highest mean CV balanced accuracy.
0.1000	0.7507	0.7453	0.0944	Good, but lower than the selected gamma.
1.0000	0.6130	0.6089	0.1091	Validation performance drops.
10.0000	0.5162	0.5000	0.0000	Near-chance validation classification.

Gamma controls the locality of the RBF kernel. Small values create smoother structure; larger values emphasize local similarity. The value was selected only from the training folds.

Gamma Sensitivity



The validation curve peaks at gamma approximately 0.0464, then declines as gamma increases further.